



National Textile University

Department of Computer Science

Subject:

Operating System

Submitted to:

Nasir Mahmood

Submitted by:

Afia Maham

Reg number:

23-NTU-CS-1126

Lab no:

09

Semester:

5th

Task01:

```
#include <stdio.h>

#include <pthread.h>

#include <semaphore.h>

#include <unistd.h>

sem_t mutex;

int counter = 0;

void* thread_function(void* arg) {

    int id = *(int*)arg;

    for (int i = 0; i < 5; i++) {

        printf("Thread %d: Waiting...\n", id);

        sem_wait(&mutex);

        counter++;

        printf("Thread %d: In critical section | Counter = %d\n", id,

            counter);

        sleep(1);

        sem_post(&mutex);

        sleep(1);

    }

    return NULL;

}

int main() {

    sem_init(&mutex, 0, 1);

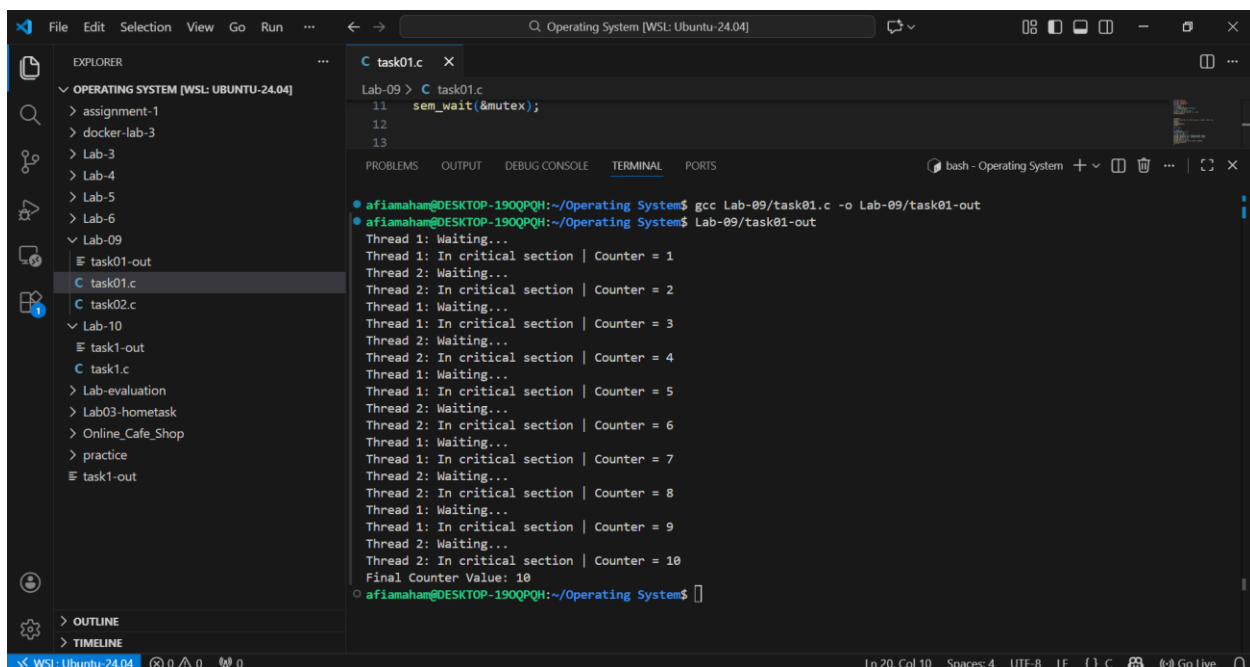
    pthread_t t1, t2;

    int id1 = 1, id2 = 2;
```

```

pthread_create(&t1, NULL, thread_function, &id1);
pthread_create(&t2, NULL, thread_function, &id2);
pthread_join(t1, NULL);
pthread_join(t2, NULL);
printf("Final Counter Value: %d\n", counter);
sem_destroy(&mutex);
return 0;
}

```



The screenshot shows the Visual Studio Code interface with a file explorer on the left and a terminal window on the right. The file explorer shows a project named 'OPERATING SYSTEM [WSL: UBUNTU-24.04]' with several subfolders and files. The terminal window shows the execution of a C program named 'task01.c'. The program uses a semaphore to synchronize two threads, 'Thread 1' and 'Thread 2', which increment a counter. The output shows the counter increasing from 1 to 10, with each thread alternating between 'Waiting...' and 'In critical section'.

```

C task01.c
Lab-09 > C task01.c
11     sem_wait(&mutex);
12
13
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
bash - Operating System
afiamaham@DESKTOP-190QPQH:~/Operating System$ gcc Lab-09/task01.c -o Lab-09/task01-out
afiamaham@DESKTOP-190QPQH:~/Operating System$ Lab-09/task01-out
Thread 1: Waiting...
Thread 1: In critical section | Counter = 1
Thread 2: Waiting...
Thread 2: In critical section | Counter = 2
Thread 1: Waiting...
Thread 1: In critical section | Counter = 3
Thread 2: Waiting...
Thread 2: In critical section | Counter = 4
Thread 1: Waiting...
Thread 1: In critical section | Counter = 5
Thread 2: Waiting...
Thread 2: In critical section | Counter = 6
Thread 1: Waiting...
Thread 1: In critical section | Counter = 7
Thread 2: Waiting...
Thread 2: In critical section | Counter = 8
Thread 1: Waiting...
Thread 1: In critical section | Counter = 9
Thread 2: Waiting...
Thread 2: In critical section | Counter = 10
Final Counter Value: 10
afiamaham@DESKTOP-190QPQH:~/Operating System$

```

Task02:

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t mutex;

int counter = 0;

```

```

void* thread_function(void* arg) {
    int id = *(int*)arg;
    for (int i = 0; i < 5; i++) {
        printf("Thread %d: Waiting...\n", id);
        sem_wait(&mutex);

        counter++;
        printf("Thread %d: In critical section | Counter = %d\n", id,
            counter);
        sleep(1);
        sem_post(&mutex);
        sleep(1);
    }
    return NULL;
}

void* thread_function2(void* arg){
    int id=*(int*)arg;
    for(int i=0;i<5;i++){
        printf("Thread %d:Waiting...\n",id);
        sem_wait(&mutex);
        counter--;
        printf("Thread %d: In critical section | Counter = %d\n", id,counter);
        sleep(1);
        sem_post(&mutex);
        sleep(1);
    }
}

```

```

        return NULL;
    }

int main() {
    sem_init(&mutex, 0, 1);
    pthread_t t1, t2;
    int id1 = 1, id2 = 2;
    pthread_create(&t1, NULL, thread_function, &id1);
    pthread_create(&t2, NULL, thread_function2, &id2);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final Counter Value: %d\n", counter);
    sem_destroy(&mutex);
    return 0;
}

```

The screenshot shows a Visual Studio Code editor window with the following components:

- EXPLORER:** A file tree on the left showing the project structure under 'OPERATING SYSTEM [WSL: UBUNTU-24.04]'. The files include 'task01.c', 'task02.c', and various output files.
- EDITOR:** The main window displays the source code for 'task02.c', which includes headers for `<stdio.h>` and `<pthread.h>`.
- TERMINAL:** The bottom panel shows the execution of the program. It starts with the command `gcc Lab-09/task02.c -o Lab-09/task02-out`. The output shows two threads, Thread 1 and Thread 2, each entering a critical section and updating a counter. The final output is 'Final Counter Value: 0'.

```

Lab-09 > C task02.c
1  #include <stdio.h>
2  #include <pthread.h>
3  #include <semaphore.h>

afiamaham@DESKTOP-190QPQH:~/Operating System$ gcc Lab-09/task02.c -o Lab-09/task02-out
afiamaham@DESKTOP-190QPQH:~/Operating System$ Lab-09/task02-out
Thread 1: Waiting...
Thread 1: In critical section | Counter = 1
Thread 2: Waiting...
Thread 2: In critical section | Counter = 0
Thread 1: Waiting...
Thread 1: In critical section | Counter = 1
Thread 2: Waiting...
Thread 2: In critical section | Counter = 0
Thread 1: Waiting...
Thread 1: In critical section | Counter = 1
Thread 2: Waiting...
Thread 2: In critical section | Counter = 0
Thread 1: Waiting...
Thread 1: In critical section | Counter = 1
Thread 2: Waiting...
Thread 2: In critical section | Counter = 0
Thread 1: Waiting...
Thread 1: In critical section | Counter = 1
Thread 2: Waiting...
Thread 2: In critical section | Counter = 0
Final Counter Value: 0
afiamaham@DESKTOP-190QPQH:~/Operating System$

```